DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

Udayapura, Kanakapura Road, Opp. Art of Living, Bangalore – 560082

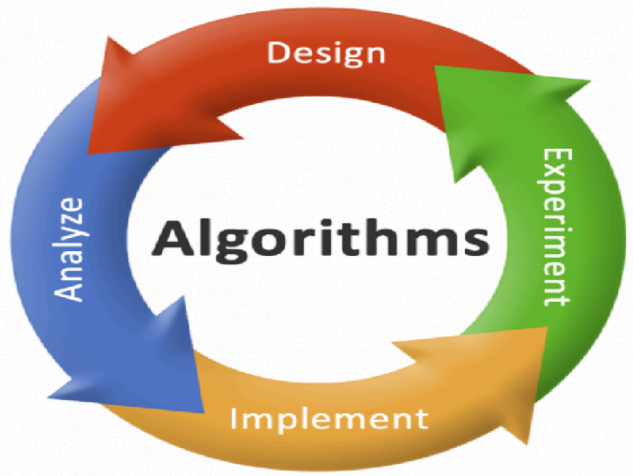
(Affliated to VTU, Belagavi, Approved by AICTE, New Delhi)

Accredited by NBA and NAAC (A+)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING 2025

DAA LABORATORY(BCS402)

LAB MANUAL



Compiled by:

Prof. Manasa Sandeep
Prof. G Yamini

Dr. Nandini C
HOD, CSE, DSATM

Dr. M Ravishankar
Principal, DSATM

INSTITUTION VISION AND MISSION

Vision of the Institution

To strive at creating the institution a center of highest caliber of learning, so as to create an overall intellectual atmosphere with each deriving strength from the other to be the best of engineers, scientists with management &design skills.

Mission of the Institution:

- To serve its region, state, the nation and globally by preparing students to make
- meaningful contributions in an increasing complex global society challenge.
- To encourage, reflection on and evaluation of emerging needs and priorities with state of art infrastructure at institution.
- To support research and services establishing enhancements in technical, health, economic, human and cultural development.
- To establish inter disciplinary center of excellence, supporting/ promoting student's implementation.
- To increase the number of Doctorate holders to promote research culture on campus.
- To establish IIPC, IPR, EDC, innovation cells with functional MOU's supporting student's quality growth.

QUALITY POLICY

Dayananda Sagar Academy of Technology and Management aims at achieving academic excellence through continuous improvement in all spheres of Technical and Management education. In pursuit of excellence cutting-edge and contemporary skills are imparted to the utmost satisfaction of the students and the concerned stakeholders

OBJECTIVES & GOALS

- Creating an academic environment to nurture and develop competent entrepreneurs, leaders and professionals who are socially sensitive and environmentally conscious.
- Integration of Outcome Based Education and cognitive teaching and learning strategies to enhance learning effectiveness.
- Developing necessary infrastructure to cater to the changing needs of Business and Society.
- Optimum utilization of the infrastructure and resources to achieve excellence in all areas of relevance.
- Adopting learning beyond curriculum through outbound activities and creative assignments.
- Imparting contemporary and emerging techno-managerial skills to keep pace with the changing global trends.
- Facilitating greater Industry-Institute Interaction for skill development and employability enhancement.
- Establishing systems and processes to facilitate research, innovation and entrepreneurship for holistic development of students.
- Implementation of Quality Assurance System in all Institutional processes

Department of Computer Science and Engineering

Vision and Mission of the Department

Department Vision

Epitomize CSE graduate to carve a niche globally in the field of computer science to excel in the world of information technology and automation by imparting knowledge to sustain skills for the changing trends in the society and industry.

Department Mission

M1: To educate students to become excellent engineers in a confident and creative environment through world-class pedagogy.

M2: Enhancing the knowledge in the changing technology trends by giving hands-on experience through continuous education and by making them to organize & participate in various events.

M3: Impart skills in the field of IT and its related areas with a focus on developing the required competencies and virtues to meet the industry expectations.

M4: Ensure quality research and innovations to fulfill industry, government & social needs.

M5: Impart entrepreneurship and consultancy skills to students to develop self-sustaining life skills in multi-disciplinary areas.

Programme Educational Objectives

PEO 1: Engage in professional practice to promote the development of innovative systems and optimized solutions for Computer Science and Engineering.

PEO 2: Adapt to different roles and responsibilities in interdisciplinary working environment by respecting professionalism and ethical practices within organization and society at national and international level.

PEO 3: Graduates will engage in life-long learning and professional development to acclimate the rapidly changing work environment and develop entrepreneurship skills.

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes:

PSO 1: Foundation of Mathematical Concepts: Ability to use mathematical methodologies to crack problem using suitable mathematical analysis, data structure and suitable algorithm.

PSO 2: Foundation of Computer System: Ability to interpret the fundamental concepts and methodology of computer systems. Students can understand the functionality of hardware and software aspects of computer systems.

PSO 3: Foundations of Software Development: Ability to grasp the software development lifecycle and methodologies of software systems. Possess competent skills and knowledge of software design process. Familiarity and practical proficiency with a broad area of programming concepts and provide new ideas and innovations towards research.

PSO 4: Foundations of Multi-Disciplinary Work: Ability to acquire leadership skills to perform professional activities with social responsibilities, through excellent flexibility to function in multi- disciplinary work environment with self-learning skills.

TABLE OF CONTENTS:

Course Outcomes: At the end of the course, the student will be able to:

CO	Course Outcomes	RBT Level	RBT Level Indicator
CO1	Define, Understand and Explain the concepts of Algorithms.	**L1/L2**	**U**
CO2	Apply the concepts of Algorithm technique in problem solving	**L3**	**Ap**
CO3	Analyze the given scenario and use appropriate algorithm to arrive at a solution	**L4**	**An**
CO4	Design and implement solution for a given problem component using Modern tools	**L6**	**C**
CO5	Evaluate the efficiency of the algorithms used in problem solving	**L5**	**E**
CO6	Conduct experiments on Data Structures using modern IDE	**L3**	**Ap**

Mapping of Course Outcomes to Program Outcomes:

CO/PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1															
CO2	**2**														
CO3		**2**													
CO4			**3**						**3**						
CO5				**2**					**2**	**2**					
CO6					**2**								**2**		

Weblinks and Video Lectures (e-Resources)	
1	http://elearning.vtu.ac.in/econtent/courses/video/CSE/06CS43.html
2	https://nptel.ac.in/courses/106/101/106101060/
3	http://elearning.vtu.ac.in/econtent/courses/video/FEP/ADA.html
4	http://cse01-iiith.vlabs.ac.in/

CIE- Continuous Internal Evaluation (50 Marks)

Bloom's Category	Theory		Practical
	Continuous Assessment Tests	Continuous Comprehensive Assessment (CCA)	Practical

	(IAT)				Test
	IAT-1	IAT-2	CCA-1	CCA-2	
	50 Marks	50 Marks	50 Marks	50 Marks	
Remember	20	10			
Understand	20	10	**10**		10
Apply	10	20	**10**		10
Analyse		10	10		10
Evaluate				10	10
Create				10	10

CIE Course Assessment Plan

CO's	Marks Distribution						Total Marks	Weightage
	Test-1			Test-2				
	Module-1	Module-2	Module 2 to 2.5	Module-2. 5 to 3	Module-4	Module-5		
CO1	**10**	**10**	**5**		**5**	**10**	**40**	**26.5%**
CO2	**10**	**10**	**5**		**5**	**10**	**40**	**26.5%**
CO3	**5**	**5**		**10**	**10**	**20**	**50**	**35%**
CO4				**5**	**5**	**10**	**20**	**14%**
CO5								
Total	**25**	**25**	**10**	**15**	**25**	**50**		

SEE- Semester End Examination (50 Marks)

Bloom's Category	SEE Marks (90% Theory+10% Practical Questions)
Remember	**10%+2%**
Understand	**20%+2%**
Apply	**40%+4%**
Analyse	**20%+2%**
Evaluate	
Create	

SEE Course Plan

CO's	Marks Distribution						Total Marks	Weightage
	Module-1	Module-2	Module 2 to 2.5	Module-2. 5 to 3	Module-4	Module-5		
CO1	**20**	**10**	**5**	**5**			**40**	**40%**
CO2	**-**	**10**	**5**	**5**	**10**	**10**	**40**	**40%**
CO3					**10**	**10**	**20**	**20%**
CO4								
CO5								

Total	20	20	10	10	20	20	100	100%

Solutions:

Sl. No.	Experiments/Programs
1	Write a program to sort a given set of elements using the Merge sort method and find the time required to sort the elements. **Solution:**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int count = 0;  // Global variable to count comparisons

// Merge function to merge two sorted subarrays
void merge(int a[], int low, int mid, int high) {
    int i, j, k;
    int c[10000]; // Temporary array

    i = low;
    j = mid + 1;
    k = 0;

    // Merge two halves into temporary array c[]
    while ((i <= mid) && (j <= high)) {
        count++; // Counting basic operations (comparisons)
```

```
        if (a[i] < a[j])
            c[k++] = a[i++];
        else
            c[k++] = a[j++];
    }

    // Copy remaining elements from left half (if any)
    while (i <= mid)
        c[k++] = a[i++];

    // Copy remaining elements from right half (if any)
    while (j <= high)
        c[k++] = a[j++];

    // Copy sorted elements back into the original array
    for (i = low, j = 0; j < k; i++, j++)
        a[i] = c[j];
}

// Merge Sort function (recursively sorts the array)
void merge_sort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2; // Find the middle index

        merge_sort(a, low,
 mid);    // Recursively sort the left half
        merge_sort(a, mid + 1, high); // Recursively sort the right half
```

```
        merge(a, low, mid, high);   // Merge the two sorted halves
    }
}

// Main function
int main() {
    int a[10000], n, i;

    // Input: Number of elements
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]); // Taking elements from the user
    }

    // Sort the array using Merge Sort
    merge_sort(a, 0, n - 1);

    // Output: Sorted array
    printf("\nAfter sorting:\n");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);



    return 0;
```

	} **Expected input and output:** Enter the number of elements: 5 Enter 5 elements: 3 1 9 0 5 Sorted Array: 0 1 3 5 9 Time taken to sort: 0.000002 seconds

2	Write a program to sort a given set of elements using Quick sort method and find the time required to sort the elements
	Code:
	
	#include <stdio.h>
	
	// Swap function without pointers
	void swap(int A[], int i, int j) {
	  int temp = A[i];
	  A[i] = A[j];
	  A[j] = temp;
	}
	
	// Partition function
	int partition(int A[], int l, int h) {
	  int pivot = A[l]; // First element as pivot
	  int i = l + 1;
	  int j = h;
	
	  while (1) {
	  // Move i to right
	  while (i <= h && A[i] < pivot)
	  i++;
	
	  // Move j to left
	  while (A[j] > pivot)
	  j--;
	
	  if (i < j) {
	  swap(A, i, j);
	  } else {
	  break;
	  }
	  }
	
	  // Place pivot in correct position

```
    swap(A, l, j);

    return j;
}

// Quicksort function
void quicksort(int A[], int l, int h) {
    if (l < h) {
        int j = partition(A, l, h);
        quicksort(A, l, j - 1);
        quicksort(A, j + 1, h);
    }
}

// Main function
int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int A[n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    quicksort(A, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    return 0;
}
```

Expected input and output:

Original Array: 0 18 -9 12 13 2 4 3 19 20
Sorted Array: -9 0 2 3 4 12 13 18 19 20
Time taken to sort: 0.000002 seconds

3	Write a program to print all the nodes reachable from a given starting node in a graph using Depth First Search method and Breadth First method. Also check connectivity of the graph. If the graph is not connected, display the number of components in the graph.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#define MAX 100
int adj[MAX][MAX], visited[MAX], n;
void depthFirstSearch(int v) {
    printf("%d ", v);
    visited[v] = 1;
    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i]) {
            depthFirstSearch(i);
        }
    }
}

void breadthFirstSearch(int start) {
    int queue[MAX], front = 0, rear = 0;
    bool visited[MAX] = {false};
        queue[rear++] = start;
    visited[start] = true;
        while (front < rear) {
        int v = queue[front++];
```

```
        printf("%d ", v);
            for (int i = 0; i < n; i++) {
          if (adj[v][i] == 1 && !visited[i]) {
            queue[rear++] = i;
            visited[i] = true;
          }
        }
    }
}
void checkGraphConnectivity() {
    int components = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            printf("Component %d: ", ++components);
            depthFirstSearch(i);
            printf("\n");
        }
    }
        if (components == 1)
        printf("The graph is connected.\n");
    else
        printf("The graph is not connected. Number of components: %d\n", components);
}
```

```
int main() {
    int start;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);
       printf("Enter the adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
        printf("Enter the starting node: ");
    scanf("%d", &start);
        printf("Reachable nodes using DFS: ");
    depthFirstSearch(start);
    printf("\n");
        printf("Reachable nodes using BFS: ");
    breadthFirstSearch(start);
    printf("\n");
        for (int i = 0; i < n; i++) visited[i] = 0;
        checkGraphConnectivity();
      return 0;
}
```

Expected input and output:

	Enter the number of nodes: 4 Enter the adjacency matrix: 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0 0 Enter the starting node: 1 Reachable nodes using DFS: 1 0 2 3 Reachable nodes using BFS: 1 0 2 3 Component 1: 0 1 2 3 The graph is connected.

4	Write a program to obtain the Topological ordering of vertices in a given digraph using
	a)Vertices deletion method
	b)DFS method
	
	Code:
	#include <stdio.h>
	#define MAX 100
	int vertices, adj[MAX][MAX];
	
	// Function to initialize graph
	void initGraph(int v) {
	  vertices = v;
	  for (int i = 0; i < vertices; i++) {
	  for (int j = 0; j < vertices; j++) {
	  adj[i][j] = 0;
	  }
	  }
	}
	
	// Function to add an edge to the graph
	void addEdge(int src, int dest) {
	  adj[src][dest] = 1;
	}

```
// Topological Sort using Vertex Deletion Method
void topologicalSortVertexDeletion() {
    int inDegree[MAX] = {0};
    int queue[MAX], front = 0, rear = 0;
    int topoOrder[MAX], index = 0;

    // Compute in-degree of each vertex
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            if (adj[i][j]) {
                inDegree[j]++;
            }
        }
    }

    // Enqueue all vertices with in-degree 0 (sources)
    for (int i = 0; i < vertices; i++) {
        if (inDegree[i] == 0) {
            queue[rear++] = i;
        }
    }

    while (front < rear) {
        int v = queue[front++];
        topoOrder[index++] = v;

        for (int i = 0; i < vertices; i++) {
            if (adj[v][i]) {
                inDegree[i]--;
                if (inDegree[i] == 0) {
                    queue[rear++] = i;
                }
            }
        }
    }

    if (index != vertices) {
        printf("Graph has a cycle! Topological sorting not possible.\n");
        return;
    }

    printf("Topological Sort (Vertex Deletion Method): ");
    for (int i = 0; i < index; i++) {
        printf("%d ", topoOrder[i]);
    }
    printf("\n");
```

```
}

// DFS function for Topological Sort
void dfs(int v, int visited[], int stack[], int *top) {
    visited[v] = 1;
    for (int i = 0; i < vertices; i++) {
        if (adj[v][i] && !visited[i]) {
            dfs(i, visited, stack, top);
        }
    }
    stack[(*top)++] = v;
}

void topologicalSortDFS() {
    int visited[MAX] = {0};
    int stack[MAX], top = 0;

    for (int i = 0; i < vertices; i++) {
        if (!visited[i]) {
            dfs(i, visited, stack, &top);
        }
    }

    printf("Topological Sort (DFS Method): ");
    while (top > 0) {
        printf("%d ", stack[--top]);
    }
    printf("\n");
}

int main() {
    int edges, src, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);
    initGraph(vertices);
    printf("Enter number of edges: ");
    scanf("%d", &edges);
    printf("Enter edges (source destination):\n");
    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &src, &dest);
        addEdge(src, dest);
    }
    topologicalSortVertexDeletion();
    topologicalSortDFS();
    return 0;
```

	} **Output**: Enter number of vertices: 4 Enter number of edges: 4 Enter edges (source destination): 1 2 0 1 0 2 0 3 Topological Sort (Vertex Deletion Method): 0 1 3 2 Topological Sort (DFS Method): 0 3 1 2

5	Write a program to sort a given set of elements using Heap sort method. Find the time complexity.
	
	Code:
	
	#include <stdio.h>
	
	#include <time.h>
	
	int c = 0, swaps = 0;
	
	void heapify(int arr[], int n, int i) {
	
	int largest = i; // Initialize largest as root
	
	int left = 2 * i + 1; // Left child
	
	int right = 2 * i + 2; // Right child
	
	// Compare left child with root
	
	if (left < n && arr[left] > arr[largest]) {
	
	largest = left;
	
	}
	
	c++;
	
	// Compare right child with largest so far
	
	if (right < n && arr[right] > arr[largest]) {
	
	largest = right;

```
    }
    c++;
    // If largest is not root, swap and continue heapifying
    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        swaps++;
        heapify(arr, n, largest); // Recursively heapify the affected subtree
    }
}
// Function to perform Heap Sort
void heapSort(int arr[], int n) {
    // Build max heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    // Extract elements one by one from heap
    for (int i = n - 1; i > 0; i--) {
        // Swap root (largest) with last element
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
```

```
        swaps++;
        // Heapify reduced heap
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    clock_t start, end;
    start = clock(); // Start time
    heapSort(arr, n);
    end = clock(); // End time
    printf("Sorted elements: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
```

```
    printf("\n");

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Time Complexity Analysis:\n");

    printf("Comparisons: %d\n", c);

    printf("Swaps: %d\n", swaps);

    printf("Time taken: %lf seconds\n", time_taken);

    return 0;

}
```

Output:

```
Enter number of elements: 5

Enter 5 elements:

3 1 0 -2 3

Sorted elements: -2 0 1 3 3

Time Complexity Analysis:

Comparisons: 18

Swaps: 7

Time taken: 0.000002 seconds
```

6 Write a program to implement Horspool's algorithm for String Matching.

Code:

```
#include<stdio.h>
#include<string.h>
#define MAX_CHAR 256
// Function to create the shift table
void createShiftTable(char pattern[], int m, int shift[]) {
    for (int i = 0; i < MAX_CHAR; i++)
        shift[i] = m;  // Default shift length
    for (int i = 0; i < m - 1; i++)
        shift[(unsigned char)pattern[i]] = m - 1 - i;
}
```

```
// Function to perform Horspool's string matching
int horspoolSearch(char text[], char pattern[]) {
    int n = strlen(text);
    int m = strlen(pattern);
    int shift[MAX_CHAR];
    createShiftTable(pattern, m, shift);
    int i = m - 1; // Start matching from the rightmost character
    while (i < n) {
        int j = 0;
        while (j < m && pattern[m - 1 - j] == text[i - j])
            j++;
        if (j == m)
            return i - m + 1; // Pattern found
        i += shift[(unsigned char)text[i]]; // Shift based on table
    }
    return -1; // Pattern not found
}
int main() {
    char text[100], pattern[50];
    printf("Enter the text: ");
    gets(text);
    printf("Enter the pattern to search: ");
    gets(pattern);
    int index = horspoolSearch(text, pattern);
    if (index != -1)
        printf("Pattern found at index %d\n", index);
    else
        printf("Pattern not found.\n");
    return 0;
}
```

Output:

```
Enter the text: design and analysis
Enter the pattern to search: and
Pattern found at index 7
```

7 Write a program to implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>
// Function to compute the maximum value in the Knapsack
void knapsack(int capacity, int weights[], int values[], int n) {
    int dp[n + 1][capacity + 1];
    // Build table dp[][] in a bottom-up manner
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0)
```

```
            dp[i][w] = 0;
        else if (weights[i - 1] <= w)
            dp[i][w] = (values[i - 1] + dp[i - 1][w - weights[i - 1]] > dp[i - 1][w]) ?
                    (values[i - 1] + dp[i - 1][w - weights[i - 1]]) : dp[i - 1][w];
        else
            dp[i][w] = dp[i - 1][w];
      }
   }
   int max_value = dp[n][capacity]; // Maximum value that can be obtained
   printf("Maximum value in Knapsack = %d\n", max_value);
   // Finding which items are included
   printf("Items included in the knapsack:\n");
   int w = capacity;
   for (int i = n; i > 0 && max_value > 0; i--) {
      if (max_value == dp[i - 1][w])
         continue;  // Item not included
      else {
         printf("Item %d (Weight: %d, Value: %d)\n", i, weights[i - 1], values[i - 1]);
         max_value -= values[i - 1];
         w -= weights[i - 1]; // Reduce weight
      }
   }
}

int main() {
   int n, capacity;
   printf("Enter the number of items: ");
   scanf("%d", &n);
   int weights[n], values[n];
   printf("Enter the weights of the items: ");
   for (int i = 0; i < n; i++)
      scanf("%d", &weights[i]);
   printf("Enter the values of the items: ");
   for (int i = 0; i < n; i++)
      scanf("%d", &values[i]);
   printf("Enter the knapsack capacity: ");
   scanf("%d", &capacity);
   knapsack(capacity, weights, values, n);

   return 0;
}
```

Output:

```
Enter the number of items: 4
Enter the weights of the items: 12 13 16 17
Enter the values of the items: 23 34 45 45
Enter the knapsack capacity: 50
```

	Maximum value in Knapsack = 124 Items included in the knapsack: Item 4 (Weight: 17, Value: 45) Item 3 (Weight: 16, Value: 45) Item 2 (Weight: 13, Value: 34)

8	Write a program to find Minimum cost spanning tree of a given undirected graph using Prim's algorithm.
	Code:
	#include<stdio.h>
	int cost[10][10],n;
	void prim()
	{
	int vt[10]={0};
	int a=0,b=0,min, mincost = 0, ne = 0;
	//start from the first vertex vt[0] = 1;
	while(ne < n-1)
	{
	//Find the nearest neighbour min = 999;
	for (int i = 0; i<n; i++)
	{
	if(vt[i]==1)
	for(int j = 0;j <n; j++)
	if(cost[i][j] < min && vt[j]==0)
	{
	min = cost[i][j];
	a = i;
	b = j;
	}
	}
	//Include nearest neighbour 'b' into MST
	printf("Edge from vertex %d to vertex %d and the cost %d\n",a,b,min); vt[b] = 1;
	ne++;
	mincost += min;
	cost[a][b] = cost[b][a] = 999;
	}
	printf("minimum spanning tree cost is %d",mincost);
	}
	void main()
	{
	printf("Enter the no. of vertices: "); scanf("%d",&n);
	printf("Enter the cost matrix\n"); for(int i=0;i<n;i++)
	for(int j=0;j<n;j++) scanf("%d",&cost[i][j]);
	prim();
	}
	Output:

	Enter the no. of vertices: 4 Enter the cost matrix 999 1 2 3 2 999 4 999 1 999 6 7 999 2 999 8 Edge from vertex 0 to vertex 0 and the cost 30183 Edge from vertex 0 to vertex 1 and the cost 1 Edge from vertex 0 to vertex 1 and the cost 1 minimum spanning tree cost is 30185

9	Write a program to find the shortest path using Dijkstra's algorithm for a weighted connected graph.
	Code:
	#include<stdio.h>
	int cost[10][10],n,dist[10];
	int minm(int m, int n)
	{
	return((m < n) ? m: n);
	}
	
	void dijkstra(int source)
	{
	int s[10]={0}; int min, w=0;
	for(int i=0;i<n;i++) dist[i]=cost[source][i];
	//Initialize dist from source to source as 0 dist[source] = 0;
	//mark source vertex - estimated for its shortest path s[source] = 1;
	for(int i=0; i < n-1; i++)
	{
	//Find the nearest neighbour vertex min = 999;
	for(int j = 0; j < n; j++)
	{
	if ((s[j] == 0) && (min > dist[j]))
	{
	min = dist[j]; w = j;
	}
	}
	s[w]=1;
	for(int v=0;v<n;v++)
	{
	if(s[v]==0 && cost[w][v]!=999)
	{
	dist[v]= minm(dist[v],dist[w]+cost[w][v]);
	}}
	}
	}

```
int main()
{
int source;
printf("Enter the no.of vertices:"); scanf("%d",&n);
printf("Enter the cost matrix\n"); for(int i=0;i<n;i++)
for(int j=0;j<n;j++) scanf("%d",&cost[i][j]);
printf("Enter the source vertex:"); scanf("%d",&source); dijkstra(source);
printf("the shortest distance is..."); for(int i=0; i<n; i++)
printf("Cost from %d to %d is %d\n",source,i,dist[i]);
}
```

Output:

```
Enter the no. of vertices:4
Enter the cost matrix
1 2 3 4
999 1 999 0
3 4 5 6
2 3 1 4
Enter the source vertex:1
the shortest distance is...Cost from 1 to 0 is 2
Cost from 1 to 1 is 1
Cost from 1 to 2 is 1
Cost from 1 to 3 is 0
```

10 Write a program to find a subset of a given set S = {S1, S2,.....,Sn} of n positive integers whose sum is equal to a given positive integer d. For example, if S ={1, 2,5, 6, 8} and d= 9, there are two solutions {1,2,6}and {1,8}. Display a suitable message, if the given problem instance doesn't have a solution.

Code:

```
#include <stdio.h>
#define MAX 100
int subset[MAX], found = 0;
// Function to find subsets with sum equal to 'd'
void findSubset(int arr[], int n, int index, int sum, int d, int subsetSize) {
   if (sum == d) {
      found = 1;
      printf("Subset found: { ");
      for (int i = 0; i < subsetSize; i++)
         printf("%d ", subset[i]);
      printf("}\n");
      return;
   }
   if (index == n || sum > d)
      return;
   // Include the current element
   subset[subsetSize] = arr[index];
   findSubset(arr, n, index + 1, sum + arr[index], d, subsetSize + 1);
```

```
    // Exclude the current element (Backtrack)
    findSubset(arr, n, index + 1, sum, d, subsetSize);
}
int main() {
    int n, d;
    printf("Enter number of elements in the set: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter the elements of the set: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Enter the target sum (d): ");
    scanf("%d", &d);
    printf("Subsets whose sum equals %d:\n", d);
    findSubset(arr, n, 0, 0, d, 0);
    if (!found)
        printf("No solution exists.\n");
    return 0;
}
```

Solution:

```
Enter number of elements in the set: 6
Enter the elements of the set: 1 2 3 4 5 6
Enter the target sum (d): 9
Subsets whose sum equals 9:
Subset found: { 1 2 6 }
Subset found: { 1 3 5 }
Subset found: { 2 3 4 }
Subset found: { 3 6 }
Subset found: { 4 5 }
```

11 Write a program to implement N -queens problem using backtracking

Code:

```
#include <stdio.h>
#define MAX 10
int board [MAX][MAX], n;
// Function to print the solution
void printSolution() {
    static int count = 1;
    printf("\nSolution %d:\n", count++);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j])
                printf(" Q ");
            else
```

```
            printf(" . ");
        }
        printf("\n");
    }
}
// Function to check if it's safe to place a queen at board[row][col]
int isSafe(int row, int col) {
    // Check column
    for (int i = 0; i < row; i++)
        if (board[i][col])
            return 0;
    // Check upper left diagonal
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return 0;
    // Check upper right diagonal
    for (int i = row, j = col; i >= 0 && j < n; i--, j++)
        if (board[i][j])
            return 0;
    return 1;
}
// Function to solve N-Queens using backtracking
void solveNQueens(int row) {
    if (row == n) { // All queens are placed
        printSolution();
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            board[row][col] = 1; // Place queen
            solveNQueens(row + 1); // Recur for next row
            board[row][col] = 0; // Backtrack
        }
    }
}
int main() {
    printf("Enter the value of N: ");
    scanf("%d", &n);
    if (n < 1 || n > MAX) {
        printf("Invalid input! Please enter a value between 1 and %d.\n", MAX);
        return 1;
    }
    printf("Solutions for %d-Queens problem:\n", n);
    solveNQueens(0);
    return 0;
}
```

Output:

Solutions for 4-Queens problem:

Solution 1:
```
. Q . .
. . . Q
Q . . .
. . Q .
```

Solution 2:
```
. . Q .
Q . . .
. . . Q
. Q . .
```

12 Write a program to solve TSP problem using branch and bound.

Code:

```
#include <stdio.h>
#include <limits.h>
#define MAX 10
int n; // Number of cities
int cost[MAX][MAX]; // Cost matrix
int min_cost = INT_MAX; // Minimum cost of the tour
int path[MAX]; // Stores the final path
int findMin(int city) {
    int min = INT_MAX;
    for (int i = 0; i < n; i++) {
        if (cost[city][i] && cost[city][i] < min)
            min = cost[city][i];
    }
    return (min == INT_MAX) ? 0 : min;
}
// Function to calculate lower bound (optimistic estimate of total cost)
int calculateBound(int visited[]) {
    int bound = 0;
    for (int i = 0; i < n; i++) {
        if (!visited[i])
            bound += findMin(i);
    }
    return bound;
}
// Recursive function to solve TSP using Branch and Bound
void tsp(int level, int current_cost, int visited[], int current_path[]) {
    if (level == n) {
        // Add cost of returning to starting city
```

```
        int total_cost = current_cost + cost[current_path[level - 1]][current_path[0]];
        if (total_cost < min_cost) {
            min_cost = total_cost;
            for (int i = 0; i < n; i++)
                path[i] = current_path[i];
        }
        return;
    }
    for (int i = 0; i < n; i++) {
        if (!visited[i] && cost[current_path[level - 1]][i]) {
            int new_cost = current_cost + cost[current_path[level - 1]][i];
            int bound = new_cost + calculateBound(visited);

            if (bound < min_cost) {
                visited[i] = 1;
                current_path[level] = i;
                tsp(level + 1, new_cost, visited, current_path);
                visited[i] = 0; // Backtrack
            }
        }
    }
}
int main() {
    printf("Enter number of cities: ");
    scanf("%d", &n);
    printf("Enter the cost matrix (use 0 for no direct path):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    int visited[MAX] = {0};
    int current_path[MAX];
    visited[0] = 1; // Start from city 0
    current_path[0] = 0;
    tsp(1, 0, visited, current_path);
    // Print the optimal path and cost
    printf("\nMinimum cost of TSP tour: %d\n", min_cost);
    printf("Optimal Path: ");
    for (int i = 0; i < n; i++)
        printf("%d -> ", path[i]);
    printf("0\n"); // Returning to the starting city
    return 0;
}
```

Output:

```
Enter number of cities: 4
Enter the cost matrix (use 0 for no direct path):
0 2 1 3
```

	4 5 6 7 12 19 10 8 1 2 09 09 Minimum cost of TSP tour: 15 Optimal Path: 0 -> 2 -> 3 -> 1 -> 0

VIVA QUESTIONS:

1. What are the properties of an algorithm?
2. How do you measure the efficiency of an algorithm?
3. What are the different types of algorithmic approaches?
4. What is the difference between worst-case, best-case, and average-case complexity?
5. Why do we prefer asymptotic analysis over absolute running time?
6. Explain the Brute Force technique. Give an example.
7. What is the time complexity of Selection Sort?
8. How does Bubble Sort work?
9. Compare Bubble Sort and Selection Sort in terms of efficiency.
10. In which cases is Bubble Sort better than Selection Sort?
11. Explain the working of Merge Sort.
12. What is the recurrence relation for Merge Sort?
13. How does QuickSort work, and what is its worst-case complexity?
14. What is the best-case time complexity of QuickSort, and when does it occur?
15. How does Strassen's Matrix Multiplication improve efficiency over the standard method?
16. How does Insertion Sort work? What is its best-case time complexity?
17. Explain the applications of DFS and BFS in graph algorithms.
18. What is Topological Sorting? Where is it used?
19. Can Topological Sorting be applied to any graph? Why or why not?
20. What is the difference between a min-heap and a max-heap?
21. Explain Counting Sort. What is its time complexity?
22. What is the basic idea behind Horspool's String-Matching Algorithm?
23. How does Boyer-Moore algorithm improve the efficiency of pattern matching?
24. What is the difference between Dynamic Programming and the Greedy approach?
25. How does Prim's Algorithm work for finding the Minimum Spanning Tree?
26. How does Dijkstra's Algorithm find the shortest path in a graph?
27. What are Huffman Trees, and how are they used in data compression?

28. Explain the Backtracking approach with an example.
29. How does the N-Queens problem use backtracking?
30. What is the Branch and Bound technique?